Solutions Guide · 해설집

Swift 자료구조 & 알고리즘 해설집

16 챕터 · 80+ 문제 완전 풀이

Array · LinkedList · Stack/Queue · Deque · Tree · BST · Heap · Graph · HashTable · Trie
Sorting · Binary Search · DP · Greedy · Backtracking · Shortest Path

목차

Ch 01 Array (배열) P1–P5

Ch 02 LinkedList (연결 리스트) P1–P5

Ch 03 Stack / Queue P1–P5

Ch 04 Deque P1–P4

Ch 05 Binary Tree (이진 트리) P1–P5

Ch 06 BST (이진 탐색 트리) P1–P5

Ch 07 Heap (힙) P1–P5

Ch 08 Graph (그래프) P1–P5

Ch 09 HashTable (해시 테이블) P1–P5

Ch 10 Trie (트라이) P1–P4

Ch 11 Sorting (정렬) P1–P5

Ch 12 Binary Search (이진 탐색) P1–P5

Ch 13 Dynamic Programming (DP) P1–P6

Ch 14 Greedy (그리디) P1–P5

Ch 15 Backtracking (백트래킹) P1–P5

Ch 16 Shortest Path (최단 경로) P1–P5

CHAPTER 01 · ARRAY

배열 (Array)

Easy

P1. 두 수의 합 (Two Sum)

> 각 원소를 보면서 target - 현재값을 해시맵에서 찾습니다. 없으면 현재값을 저장하고 다음으로 넘어갑니다. 한 번의 순회로 O(n) 시간에 해결됩니다.

```
func twoSum(_ nums: [Int], _ target: Int) -> [Int] {
    var map: [Int: Int] = [:]  // 값 → 인덱스
    for (i, n) in nums.enumerated() {
        if let j = map[target - n] { return [j, i] }
        map[n] = i
    }
    return []
}
```

시간: O(n) | 공간: O(n)

Easy

P2. 최대 연속 부분합 (Maximum Subarray) — Kadane's Algorithm

> 현재까지의 합(cur)이 음수가 되면 버리고 현재 원소에서 새로 시작합니다. 매 단계에서 전역 최댓값을 갱신합니다.

```
func maxSubArray(_ nums: [Int]) -> Int {
    var cur = nums[0], best = nums[0]
    for i in 1..<nums.count {
        cur = max(nums[i], cur + nums[i])  // 이어가거나 새로 시작
        best = max(best, cur)
    }
    return best
}
```

시간: O(n) | 공간: O(1)

Medium

P3. 제품 배열 (Product Except Self)

> 왼쪽 누적 곱 배열을 결과로 두고, 오른쪽에서 한 번 더 순회하며 곱합니다. 나눗셈 없이 O(n) 시간, O(1) 추가 공간으로 해결됩니다.

```
func productExceptSelf(_ nums: [Int]) -> [Int] {
    let n = nums.count
    var res = Array(repeating: 1, count: n)
    var left = 1
    for i in 0..<n { res[i] = left; left *= nums[i] }
    var right = 1
    for i in stride(from: n-1, through: 0, by: -1) {
        res[i] *= right; right *= nums[i]
    }
    return res
}
```

시간: O(n) | 공간: O(1) 추가공간

Medium

P4. 컨테이너 물 채우기 (Container With Most Water)

> 양쪽 끝에서 포인터를 좁혀가며, 더 낮은 쪽을 이동합니다. 높은 쪽을 이동하면 넓이가 줄기만 하므로 낮은 쪽을 이동하는 것이 최적입니다.

```
func maxWater(_ height: [Int]) -> Int {
    var lo = 0, hi = height.count - 1, best = 0
    while lo < hi {
        best = max(best, min(height[lo], height[hi]) * (hi - lo))
        if height[lo] < height[hi] { lo += 1 } else { hi -= 1 }
    }
    return best
}
```

시간: O(n) | 공간: O(1)

Hard

P5. 트래핑 빗물 (Trapping Rain Water)

> 투 포인터로 maxLeft, maxRight를 추적합니다. 작은 쪽이 병목이므로 그쪽에서 물을 계산하고 포인터를 이동합니다.

```
func trap(_ height: [Int]) -> Int {
    var lo = 0, hi = height.count - 1
    var maxL = 0, maxR = 0, water = 0
    while lo < hi {
        if height[lo] < height[hi] {
            if height[lo] >= maxL { maxL = height[lo] }
            else { water += maxL - height[lo] }
            lo += 1
        } else {
            if height[hi] >= maxR { maxR = height[hi] }
            else { water += maxR - height[hi] }
            hi -= 1
        }
    }
    return water
}
```

시간: O(n) | 공간: O(1)

CHAPTER 02 · LINKEDLIST

연결 리스트 (Linked List)

Easy

P1. 연결 리스트 뒤집기 (Reverse Linked List)

> prev/curr/next 세 포인터를 이용한 반복문으로 각 노드의 next 방향을 역전시킵니다. 재귀보다 스택 오버플로우 위험이 없습니다.

```
func reverseList(_ head: ListNode?) -> ListNode? {
    var prev: ListNode? = nil, curr = head
    while let c = curr {
        let next = c.next
        c.next = prev
        prev = c
        curr = next
    }
    return prev
}
```

시간: O(n) | 공간: O(1)

Easy

P2. 중간 노드 찾기 (Middle of the Linked List)

> slow는 1칸, fast는 2칸 이동합니다. fast가 끝에 닿으면 slow가 정확히 중간 노드를 가리킵니다.

```
func middleNode(_ head: ListNode?) -> ListNode? {
    var slow = head, fast = head
    while fast?.next != nil {
        slow = slow?.next
        fast = fast?.next?.next
    }
    return slow
}
```

시간: O(n) | 공간: O(1)

Medium

P3. 두 정렬 리스트 합치기 (Merge Two Sorted Lists)

> dummy 노드를 앞에 두고 두 포인터 중 작은 값을 연결합니다. dummy.next로 결과 헤드를 반환합니다.

```
func mergeTwoLists(_ l1: ListNode?, _ l2: ListNode?) -> ListNode? {
    let dummy = ListNode(0)
    var cur: ListNode? = dummy, a = l1, b = l2
    while let x = a, let y = b {
        if x.val <= y.val { cur?.next = x; a = x.next }
        else              { cur?.next = y; b = y.next }
        cur = cur?.next
    }
    cur?.next = a ?? b
    return dummy.next
}
```

시간: O(m+n) | 공간: O(1)

🟡 Medium

P4. 사이클 감지 (Linked List Cycle) — Floyd's Algorithm

> slow 1칸, fast 2칸 이동합니다. 사이클이 있으면 두 포인터는 반드시 만납니다. 추가 공간 없이 O(1)로 감지합니다.

```
func hasCycle(_ head: ListNode?) -> Bool {
    var slow = head, fast = head
    while fast?.next != nil {
        slow = slow?.next
        fast = fast?.next?.next
        if slow === fast { return true }
    }
    return false
}
```

시간: O(n) | 공간: O(1)

Hard

P5. k개씩 뒤집기 (Reverse Nodes in k-Group)

k개씩 그룹을 세어 뒤집고 재귀로 나머지를 처리합니다. k개가 남지 않으면 그대로 반환합니다.

```
func reverseKGroup(_ head: ListNode?, _ k: Int) -> ListNode? {
    var count = 0; var node = head
    while let n = node, count < k { node = n.next; count += 1 }
    guard count == k else { return head }
    var prev: ListNode? = reverseKGroup(node, k), curr = head
    for _ in 0..<k {
        let next = curr?.next; curr?.next = prev; prev = curr; curr =
    }
    return prev
}
```

시간: O(n) | 공간: O(n/k) 재귀

CHAPTER 03 · STACK / QUEUE

스택 / 큐

Easy

P1. 유효한 괄호 (Valid Parentheses)

> 여는 괄호는 스택에 push, 닫는 괄호는 스택 top과 매핑 테이블로 비교합니다. 마지막에 스택이 비어있으면 유효합니다.

```
func isValid(_ s: String) -> Bool {
    var stack: [Character] = []
    let map: [Character: Character] = [")": "(", "]": "[", "}": "{"]
    for c in s {
        if "([{".contains(c) { stack.append(c) }
        else if stack.last != map[c] { return false }
        else { stack.removeLast() }
    }
    return stack.isEmpty
}
```

시간: O(n) | 공간: O(n)

Easy

P2. 일일 온도 (Daily Temperatures) — 단조 감소 스택

> 인덱스를 스택에 저장하며, 현재 온도가 스택 top 온도보다 높으면 pop하여 대기 일수를 계산합니다. 단조 감소 스택 패턴입니다.

```
func dailyTemperatures(_ t: [Int]) -> [Int] {
    var res = Array(repeating: 0, count: t.count), stack: [Int] = []
    for i in 0..<t.count {
        while let top = stack.last, t[i] > t[top] {
            res[top] = i - top; stack.removeLast()
        }
        stack.append(i)
    }
    return res
}
```

시간: O(n) | 공간: O(n)

🟡 Medium

P3. 스택 두 개로 큐 구현 (Implement Queue using Stacks)

> inbox(입력용)와 outbox(출력용) 스택을 사용합니다. outbox가 비면 inbox의 모든 원소를 옮깁니다. amortized O(1) 연산을 보장합니다.

```
class MyQueue {
    var inbox: [Int] = [], outbox: [Int] = []
    func push(_ x: Int) { inbox.append(x) }
    func pop() -> Int { peek(); return outbox.removeLast() }
    func peek() -> Int {
        if outbox.isEmpty {
            while !inbox.isEmpty { outbox.append(inbox.removeLast()) }
        }
        return outbox.last!
    }
    func empty() -> Bool { return inbox.isEmpty && outbox.isEmpty }
}
```

시간: amortized O(1) | 공간: O(n)

Medium

P4. 최솟값 O(1) 스택 (Min Stack)

> 각 원소를 push할 때 현재 최솟값도 함께 튜플로 저장합니다. pop해도 이전 최솟값이 보존됩니다.

```
class MinStack {
    var stack: [(val: Int, min: Int)] = []
    func push(_ val: Int) {
        let m = stack.isEmpty ? val : min(stack.last!.min, val)
        stack.append((val, m))
    }
    func pop() { stack.removeLast() }
    func top() -> Int { stack.last!.val }
    func getMin() -> Int { stack.last!.min }
}
```

시간: O(1) 모든 연산 | 공간: O(n)

🔴 Hard

P5. 슬라이딩 윈도우 최댓값 (Sliding Window Maximum) — 단조 덱

> 단조 감소 덱에 인덱스를 저장합니다. 앞에서 윈도우 범위를 벗어난 인덱스를 제거하고, 뒤에서 현재값보다 작은 원소를 제거합니다.

```
func maxSlidingWindow(_ nums: [Int], _ k: Int) -> [Int] {
    var deque: [Int] = [], res: [Int] = []
    for i in 0..<nums.count {
        // 윈도우 범위 벗어난 인덱스 제거
        while !deque.isEmpty && deque.first! < i - k + 1 { deque.remov
        // 현재값보다 작은 원소 뒤에서 제거 (단조 감소 유지)
        while !deque.isEmpty && nums[deque.last!] < nums[i] { deque.re
        deque.append(i)
        if i >= k - 1 { res.append(nums[deque.first!]) }
    }
    return res
}
```

시간: O(n) | 공간: O(k)

CHAPTER 04 · DEQUE

덱 (Deque)

Easy

P1. 팰린드롬 확인 (Valid Palindrome)

> 문자열을 필터링해 배열로 만들고, 양쪽 끝에서 하나씩 꺼내 비교합니다. 덱의 양방향 접근을 활용합니다.

```
func isPalindrome(_ s: String) -> Bool {
    let filtered = s.lowercased().filter { $0.isLetter || $0.isNumber
    var deque = Array(filtered)
    while deque.count > 1 {
        if deque.removeFirst() != deque.removeLast() { return false }
    }
    return true
}
```

시간: O(n) | 공간: O(n)

Medium

P2. 이동 평균 (Moving Average from Data Stream)

> 고정 크기 큐로 슬라이딩 윈도우를 구현합니다. 윈도우가 가득 차면 가장 오래된 원소를 제거하고 합계를 갱신합니다.

```
class MovingAverage {
    let size: Int; var queue: [Int] = []; var sum = 0
    init(_ size: Int) { self.size = size }
    func next(_ val: Int) -> Double {
        if queue.count == size { sum -= queue.removeFirst() }
        queue.append(val); sum += val
        return Double(sum) / Double(queue.count)
    }
}
```

시간: O(1) | 공간: O(k)

Medium

P3. 원형 덱 구현 (Design Circular Deque)

> 고정 크기 배열과 front/rear 인덱스를 모듈로 연산으로 관리합니다. count로 비어있음/가득 참을 추적합니다.

```
class MyCircularDeque {
    var buf: [Int]; var front = 0, rear = 0, count = 0, cap: Int
    init(_ k: Int) { cap = k; buf = Array(repeating: 0, count: k) }
    func insertFront(_ v: Int) -> Bool {
        guard !isFull() else { return false }
        front = (front - 1 + cap) % cap; buf[front] = v; count += 1; r
    }
    func insertLast(_ v: Int) -> Bool {
        guard !isFull() else { return false }
        buf[rear] = v; rear = (rear + 1) % cap; count += 1; return tru
    }
    func deleteFront() -> Bool {
        guard !isEmpty() else { return false }
        front = (front + 1) % cap; count -= 1; return true
    }
    func deleteLast() -> Bool {
        guard !isEmpty() else { return false }
        rear = (rear - 1 + cap) % cap; count -= 1; return true
    }
    func getFront() -> Int { isEmpty() ? -1 : buf[front] }
    func getRear() -> Int { isEmpty() ? -1 : buf[(rear - 1 + cap) % ca
    func isEmpty() -> Bool { count == 0 }
    func isFull() -> Bool { count == cap }
}
```

시간: O(1) | 공간: O(k)

Hard

P4. 가장 긴 유효 괄호 (Longest Valid Parentheses)

> 스택에 -1 센티넬을 두고 인덱스를 관리합니다. 닫는 괄호가 나올 때 pop 후 스택이 비면 현재 인덱스를 센티넬로 push, 아니면 현재 인덱스와 top의 차이가 유효 길이입니다.

```
func longestValidParentheses(_ s: String) -> Int {
    var stack = [-1], best = 0  // -1 센티넬
    for (i, c) in s.enumerated() {
        if c == "(" {
            stack.append(i)
        } else {
            stack.removeLast()
            if stack.isEmpty { stack.append(i) }  // 새 센티넬
            else { best = max(best, i - stack.last!) }
        }
    }
    return best
}
```

시간: O(n) | 공간: O(n)

CHAPTER 05 · BINARY TREE

이진 트리 (Binary Tree)

Easy

P1. 최대 깊이 (Maximum Depth of Binary Tree)

재귀적으로 왼쪽/오른쪽 서브트리의 깊이를 구하고 1을 더합니다. nil이면 0을 반환하는 베이스 케이스입니다.

```
func maxDepth(_ root: TreeNode?) -> Int {
    guard let root = root else { return 0 }
    return 1 + max(maxDepth(root.left), maxDepth(root.right))
}
```

시간: O(n) | 공간: O(h)

Easy

P2. 대칭 확인 (Symmetric Tree)

> 거울 함수를 만들어 왼쪽 트리의 왼쪽과 오른쪽 트리의 오른쪽, 왼쪽 트리의 오른쪽과 오른쪽 트리의 왼쪽을 재귀적으로 비교합니다.

```
func isSymmetric(_ root: TreeNode?) -> Bool {
    func mirror(_ a: TreeNode?, _ b: TreeNode?) -> Bool {
        guard let a = a, let b = b else { return a == nil && b == nil
        return a.val == b.val
            && mirror(a.left, b.right)
            && mirror(a.right, b.left)
    }
    return mirror(root?.left, root?.right)
}
```

시간: O(n) | 공간: O(h)

🟡 Medium

P3. 레벨 순서 순회 (Binary Tree Level Order Traversal)

> BFS 큐에 루트부터 담고, 한 레벨 전체를 한 번에 처리합니다. flatMap으로 다음 레벨 노드를 모읍니다.

```
func levelOrder(_ root: TreeNode?) -> [[Int]] {
    guard let root = root else { return [] }
    var res: [[Int]] = [], queue = [root]
    while !queue.isEmpty {
        let level = queue.map { $0.val }
        res.append(level)
        queue = queue.flatMap { [$0.left, $0.right].compactMap { $0 }
    }
    return res
}
```

시간: O(n) | 공간: O(n)

🟡 Medium

P4. 경로 합 (Path Sum)

> 리프 노드에서 남은 target이 노드값과 같으면 경로가 존재합니다. 매 단계에서 target에서 현재 노드값을 빼며 재귀합니다.

```
func hasPathSum(_ root: TreeNode?, _ target: Int) -> Bool {
    guard let root = root else { return false }
    if root.left == nil && root.right == nil { return root.val == targ
    return hasPathSum(root.left, target - root.val)
        || hasPathSum(root.right, target - root.val)
}
```

시간: O(n) | 공간: O(h)

Hard

P5. 최대 경로 합 (Binary Tree Maximum Path Sum)

> 각 노드에서 왼쪽/오른쪽 기여값(음수면 0)을 구하고, 이 노드를 지나는 경로 합으로 전역 최댓값을 갱신합니다. 부모로는 한 방향의 최댓값만 전달합니다.

```
func maxPathSum(_ root: TreeNode?) -> Int {
    var best = Int.min
    @discardableResult
    func dfs(_ node: TreeNode?) -> Int {
        guard let node = node else { return 0 }
        let l = max(0, dfs(node.left))
        let r = max(0, dfs(node.right))
        best = max(best, node.val + l + r)  // 이 노드를 지나는 최대 경로
        return node.val + max(l, r)          // 부모로 전달할 수 있는 최댓값
    }
    dfs(root)
    return best
}
```

시간: O(n) | 공간: O(h)

CHAPTER 06 · BST

이진 탐색 트리 (BST)

Easy

P1. BST 유효성 검사 (Validate Binary Search Tree)

> 각 노드에 허용 범위(min, max)를 전달합니다. 왼쪽으로 내려갈 때는 max를 현재값으로, 오른쪽으로 내려갈 때는 min을 현재값으로 갱신합니다.

```
func isValidBST(_ root: TreeNode?) -> Bool {
    func check(_ node: TreeNode?, _ min: Int, _ max: Int) -> Bool {
        guard let node = node else { return true }
        guard node.val > min && node.val < max else { return false }
        return check(node.left, min, node.val)
            && check(node.right, node.val, max)
    }
    return check(root, Int.min, Int.max)
}
```

시간: O(n) | 공간: O(h)

Easy

P2. k번째 최솟값 (Kth Smallest Element in a BST)

> 중위 순회(inorder)는 BST를 오름차순으로 방문합니다. count를 감소시키다가 0이 되는 순간의 노드값이 k번째 최솟값입니다.

```
func kthSmallest(_ root: TreeNode?, _ k: Int) -> Int {
    var count = k, result = 0
    func inorder(_ node: TreeNode?) {
        guard let node = node, count > 0 else { return }
        inorder(node.left)
        count -= 1
        if count == 0 { result = node.val; return }
        inorder(node.right)
    }
    inorder(root)
    return result
}
```

시간: O(h+k) | 공간: O(h)

Medium

P3. 두 노드 합 = k (Two Sum IV - Input is a BST)

> DFS로 순회하며 k - 현재값이 이미 seen에 있으면 true를 반환합니다. HashSet으로 O(n) 시간에 해결합니다.

```
func findTarget(_ root: TreeNode?, _ k: Int) -> Bool {
    var seen = Set<Int>()
    func dfs(_ node: TreeNode?) -> Bool {
        guard let node = node else { return false }
        if seen.contains(k - node.val) { return true }
        seen.insert(node.val)
        return dfs(node.left) || dfs(node.right)
    }
    return dfs(root)
}
```

시간: O(n) | 공간: O(n)

🟡 Medium

P4. 정렬 배열 → 균형 BST (Convert Sorted Array to BST)

> 중간 인덱스를 루트로, 왼쪽 절반은 왼쪽 서브트리, 오른쪽 절반은 오른쪽 서브트리로 재귀합니다. 항상 높이 균형 트리가 만들어집니다.

```
func sortedArrayToBST(_ nums: [Int]) -> TreeNode? {
    guard !nums.isEmpty else { return nil }
    let mid = nums.count / 2
    let node = TreeNode(nums[mid])
    node.left  = sortedArrayToBST(Array(nums[..<mid]))
    node.right = sortedArrayToBST(Array(nums[(mid+1)...]))
    return node
}
```

시간: O(n) | 공간: O(n)

🔴 Hard

P5. Greater Sum Tree 변환 (Convert BST to Greater Tree)

> 역중위 순회(오른쪽 → 루트 → 왼쪽)로 큰 값부터 방문합니다. 누적 합을 현재 노드값에 더하면 자신보다 크거나 같은 값의 합이 됩니다.

```
func convertBST(_ root: TreeNode?) -> TreeNode? {
    var sum = 0
    func reverseInorder(_ node: TreeNode?) {
        guard let node = node else { return }
        reverseInorder(node.right)   // 먼저 오른쪽(큰 값들) 처리
        sum += node.val
        node.val = sum
        reverseInorder(node.left)
    }
    reverseInorder(root)
    return root
}
```

시간: O(n) | 공간: O(h)

CHAPTER 07 · HEAP

힙 (Heap)

Easy

P1. K번째로 큰 원소 (Kth Largest Element in an Array)

> 내림차순 정렬 후 k-1 인덱스를 반환합니다. 최적화 방법은 크기 k의 Min Heap을 유지하면 O(n log k)로 줄일 수 있습니다.

```
func findKthLargest(_ nums: [Int], _ k: Int) -> Int {
    return nums.sorted(by: >)[k-1]
    // 최적: 크기 k Min Heap 유지 → O(n log k)
}
```

시간: O(n log n) | 공간: O(1)

Easy

P2. K개 빈번한 원소 (Top K Frequent Elements)

딕셔너리로 빈도를 계산하고, 빈도 내림차순으로 정렬 후 상위 k개를 반환합니다. Bucket Sort로 O(n)에도 해결 가능합니다.

```
func topKFrequent(_ nums: [Int], _ k: Int) -> [Int] {
    var freq: [Int: Int] = [:]
    nums.forEach { freq[$0, default: 0] += 1 }
    return freq.sorted { $0.value > $1.value }
               .prefix(k)
               .map { $0.key }
}
```

시간: O(n log n) | 공간: O(n)

🟡 Medium

P3. 막대 연결 최소 비용 (Minimum Cost to Connect Sticks)

> 항상 가장 짧은 두 막대를 연결해야 비용이 최소입니다. Min Heap에서 두 개씩 꺼내 합산하고 다시 넣습니다.

```
func connectSticks(_ sticks: [Int]) -> Int {
    // Swift에 내장 Heap이 없으므로 정렬 배열로 시뮬레이션
    // 실제 코딩테스트에서는 우선순위 큐 라이브러리 사용 권장
    var heap = sticks.sorted()
    var cost = 0
    while heap.count > 1 {
        heap.sort()  // Min Heap 역할
        let combined = heap.removeFirst() + heap.removeFirst()
        cost += combined
        heap.append(combined)
    }
    return cost
}
```

시간: O(n log n) | 공간: O(n)

Hard

P4. 중간값 스트림 (Find Median from Data Stream)

> Max Heap(왼쪽 절반)과 Min Heap(오른쪽 절반)으로 분리합니다. 두 힙의 크기 차이가 1 이하가 되도록 균형을 유지하면, top 값들로 중간값을 O(1)에 구합니다.

```
// Max Heap (왼쪽 절반) + Min Heap (오른쪽 절반)
class MedianFinder {
    var lower: [Int] = []  // 내림차순 (max heap 시뮬레이션)
    var upper: [Int] = []  // 오름차순 (min heap 시뮬레이션)
    func addNum(_ num: Int) {
        lower.append(num); lower.sort(by: >)
        if !upper.isEmpty && lower[0] > upper[0] {
            upper.append(lower.removeFirst()); upper.sort()
        }
        if lower.count > upper.count + 1 {
            upper.append(lower.removeFirst()); upper.sort()
        } else if upper.count > lower.count {
            lower.append(upper.removeFirst()); lower.sort(by: >)
        }
    }
    func findMedian() -> Double {
        if lower.count == upper.count {
            return Double(lower[0] + upper[0]) / 2.0
        }
        return Double(lower[0])
    }
}
```

시간: O(n log n) 삽입 | 공간: O(n)

Hard

P5. K개 정렬 리스트 병합 (Merge K Sorted Lists)

> 분할 정복으로 2개씩 병합을 반복합니다. 매 라운드 절반으로 줄어 O(log k) 라운드, 각 라운드 O(n)으로 전체 O(n log k)입니다.

```
func mergeKLists(_ lists: [ListNode?]) -> ListNode? {
    var lists = lists.compactMap { $0 }
    while lists.count > 1 {
        var merged: [ListNode] = []
        for i in stride(from: 0, to: lists.count, by: 2) {
            let a: ListNode? = lists[i]
            let b: ListNode? = i+1 < lists.count ? lists[i+1] : nil
            if let m = merge2(a, b) { merged.append(m) }
        }
        lists = merged
    }
    return lists.first
}
func merge2(_ a: ListNode?, _ b: ListNode?) -> ListNode? {
    let d = ListNode(0); var cur: ListNode? = d, x = a, y = b
    while let p = x, let q = y {
        if p.val <= q.val { cur?.next = p; x = p.next }
        else { cur?.next = q; y = q.next }
        cur = cur?.next
    }
    cur?.next = x ?? y; return d.next
}
```

시간: O(n log k) | 공간: O(log k) 재귀

CHAPTER 08 · GRAPH

그래프 (Graph)

Easy

P1. 섬의 개수 (Number of Islands)

'1'을 발견하면 count를 늘리고 DFS로 연결된 모든 '1'을 '0'으로 방문 처리합니다. 격자를 직접 수정해 visited 배열을 절약합니다.

```
func numIslands(_ grid: [[Character]]) -> Int {
    var grid = grid, count = 0
    func dfs(_ r: Int, _ c: Int) {
        guard r >= 0 && r < grid.count
           && c >= 0 && c < grid[0].count
           && grid[r][c] == "1" else { return }
        grid[r][c] = "0"
        dfs(r+1,c); dfs(r-1,c); dfs(r,c+1); dfs(r,c-1)
    }
    for r in 0..<grid.count {
        for c in 0..<grid[0].count {
            if grid[r][c] == "1" { dfs(r, c); count += 1 }
        }
    }
    return count
}
```

시간: O(m×n) | 공간: O(m×n)

🟡 Medium

P2. 코스 수강 가능 여부 (Course Schedule) — 위상 정렬

> 진입 차수(inDegree)가 0인 노드부터 BFS로 처리합니다. 완료 노드 수가 전체와 같으면 사이클이 없어 수강 가능합니다.

```
func canFinish(_ n: Int, _ pre: [[Int]]) -> Bool {
    var adj = Array(repeating: [Int](), count: n)
    var inDegree = Array(repeating: 0, count: n)
    for p in pre { adj[p[1]].append(p[0]); inDegree[p[0]] += 1 }
    var queue = (0..<n).filter { inDegree[$0] == 0 }, done = 0
    while !queue.isEmpty {
        let u = queue.removeFirst(); done += 1
        for v in adj[u] {
            inDegree[v] -= 1
            if inDegree[v] == 0 { queue.append(v) }
        }
    }
    return done == n
}
```

시간: O(V+E) | 공간: O(V+E)

🟡 Medium

P3. 연결 요소 수 (Number of Connected Components) — Union-Find

> Union-Find(path compression)로 엣지를 합칩니다. 서로 다른 컴포넌트를 합칠 때 count를 줄입니다. 최종 count가 연결 요소의 수입니다.

```
func countComponents(_ n: Int, _ edges: [[Int]]) -> Int {
    var parent = Array(0..<n)
    func find(_ x: Int) -> Int {
        if parent[x] != x { parent[x] = find(parent[x]) }
        return parent[x]
    }
    var count = n
    for e in edges {
        let a = find(e[0]), b = find(e[1])
        if a != b { parent[a] = b; count -= 1 }
    }
    return count
}
```

시간: O(n α(n)) | 공간: O(n)

Hard

P4. 단어 사다리 (Word Ladder) — BFS

> BFS로 한 글자씩 변경해 도달 가능한 단어를 탐색합니다. 방문한 단어를 set에서 제거해 재방문을 방지합니다. 레벨 수 = 최단 변환 횟수입니다.

```
func ladderLength(_ begin: String, _ end: String,
                  _ wordList: [String]) -> Int {
    var wordSet = Set(wordList)
    guard wordSet.contains(end) else { return 0 }
    var queue = [begin], steps = 1
    let alpha = "abcdefghijklmnopqrstuvwxyz"
    while !queue.isEmpty {
        steps += 1
        var next: [String] = []
        for word in queue {
            var chars = Array(word)
            for i in 0..<chars.count {
                let orig = chars[i]
                for c in alpha {
                    chars[i] = c
                    let nw = String(chars)
                    if nw == end { return steps }
                    if wordSet.contains(nw) {
                        next.append(nw); wordSet.remove(nw)
                    }
                }
                chars[i] = orig
            }
        }
        queue = next
    }
    return 0
}
```

시간: O(n × m × 26) | 공간: O(n)

Hard

P5. 외계 언어 사전 (Alien Dictionary) — 위상 정렬

> 인접한 단어들을 비교해 첫 번째로 다른 글자 쌍에서 순서 규칙을 추출합니다. 규칙들을 위상 정렬하면 알파벳 순서가 나옵니다.

```
func alienOrder(_ words: [String]) -> String {
    var adj: [Character: Set<Character>] = [:]
    var inDeg: [Character: Int] = [:]
    for w in words { for c in w {
        if inDeg[c] == nil { inDeg[c] = 0; adj[c] = [] }
    }}
    for i in 0..<words.count-1 {
        let a = Array(words[i]), b = Array(words[i+1])
        if a.count > b.count && a.starts(with: b) { return "" }
        for j in 0..<min(a.count, b.count) {
            if a[j] != b[j] {
                if !adj[a[j]]!.contains(b[j]) {
                    adj[a[j]]!.insert(b[j]); inDeg[b[j]]! += 1
                }
                break
            }
        }
    }
    var queue = inDeg.filter { $0.value == 0 }.map { $0.key }.sorted()
    var result: [Character] = []
    while !queue.isEmpty {
        let c = queue.removeFirst(); result.append(c)
        for next in adj[c]!.sorted() {
            inDeg[next]! -= 1
            if inDeg[next]! == 0 { queue.append(next); queue.sort() }
        }
    }
    return result.count == inDeg.count ? String(result) : ""
}
```

시간: O(C) C=전체 문자 수 | 공간: O(1) (26글자)

CHAPTER 09 · HASHTABLE

해시 테이블 (Hash Table)

Easy

P1. 애너그램 그룹화 (Group Anagrams)

> 각 문자열을 정렬하면 애너그램끼리 같은 키를 가집니다. 딕셔너리로 그룹화한 후 values를 반환합니다.

```
func groupAnagrams(_ strs: [String]) -> [[String]] {
    var map: [String: [String]] = [:]
    for s in strs {
        let key = String(s.sorted())
        map[key, default: []].append(s)
    }
    return Array(map.values)
}
```

시간: O(n × m log m) | 공간: O(n)

Medium

P2. 최장 연속 수열 (Longest Consecutive Sequence)

> Set에 모든 수를 넣고, n-1이 없는 수(시작점)에서만 연속 길이를 셉니다. 각 원소는 최대 2번 접근되므로 O(n)입니다.

```
func longestConsecutive(_ nums: [Int]) -> Int {
    let set = Set(nums); var best = 0
    for n in set where !set.contains(n - 1) {  // 시작점만 탐색
        var cur = n, len = 1
        while set.contains(cur + 1) { cur += 1; len += 1 }
        best = max(best, len)
    }
    return best
}
```

시간: O(n) | 공간: O(n)

Medium

P3. 부분배열 합 = k (Subarray Sum Equals K)

> prefix sum을 계산하며, 이전에 나온 prefix sum들을 해시맵에 저장합니다. prefix - k가 이전에 있었다면 그만큼의 부분배열이 존재합니다.

```
func subarraySum(_ nums: [Int], _ k: Int) -> Int {
    var map = [0: 1], prefix = 0, count = 0
    for n in nums {
        prefix += n
        count += map[prefix - k, default: 0]
        map[prefix, default: 0] += 1
    }
    return count
}
```

시간: O(n) | 공간: O(n)

🟡 Medium

P4. 최장 중복 없는 부분문자열 (Longest Substring Without Repeating Characters)

> 슬라이딩 윈도우 기법으로, 중복 문자가 나오면 start를 그 문자의 이전 위치 + 1로 이동합니다. 매 단계에서 윈도우 크기의 최댓값을 갱신합니다.

```
func lengthOfLongestSubstring(_ s: String) -> Int {
    var map: [Character: Int] = [:], start = 0, best = 0
    for (i, c) in s.enumerated() {
        if let prev = map[c], prev >= start { start = prev + 1 }
        map[c] = i
        best = max(best, i - start + 1)
    }
    return best
}
```

시간: O(n) | 공간: O(min(n, m)) m=문자 집합 크기

🔴 Hard

P5. 최소 윈도우 부분문자열 (Minimum Window Substring)

> 슬라이딩 윈도우로 t의 모든 문자를 포함하는 최소 구간을 찾습니다. have/total 카운터로 조건 충족 여부를 O(1)에 확인합니다.

```
func minWindow(_ s: String, _ t: String) -> String {
    var need: [Character: Int] = [:]
    t.forEach { need[$0, default: 0] += 1 }
    var window: [Character: Int] = [:], have = 0, total = need.count
    var res = "", resLen = Int.max
    var lo = s.startIndex
    for (hi, c) in zip(s.indices, s) {
        window[c, default: 0] += 1
        if need[c] != nil && window[c] == need[c] { have += 1 }
        while have == total {
            let len = s.distance(from: lo, to: hi) + 1
            if len < resLen { resLen = len; res = String(s[lo...hi]) }
            let lc = s[lo]
            window[lc, default: 0] -= 1
            if need[lc] != nil && window[lc]! < need[lc]! { have -= 1
            lo = s.index(after: lo)
        }
    }
    return res
}
```

시간: O(n+m) | 공간: O(m)

CHAPTER 10 · TRIE

트라이 (Trie)

Easy

P1. Trie 구현 (Implement Trie)

> 각 노드는 자식 딕셔너리와 isEnd 플래그를 가집니다. insert/search/startsWith 모두 문자를 하나씩 따라가며 노드를 탐색합니다.

```
final class Trie {
    final class Node {
        var children = [Character: Node]()
        var isEnd = false
    }
    let root = Node()
    func insert(_ word: String) {
        var cur = root
        for c in word {
            if cur.children[c] == nil { cur.children[c] = Node() }
            cur = cur.children[c]!
        }
        cur.isEnd = true
    }
    func search(_ word: String) -> Bool { node(word)?.isEnd == true }
    func startsWith(_ prefix: String) -> Bool { node(prefix) != nil }
    private func node(_ s: String) -> Node? {
        var cur: Node? = root
        for c in s { cur = cur?.children[c] }
        return cur
    }
}
```

시간: O(m) 각 연산 | 공간: O(ALPHABET × m × n)

Medium

P2. 단어 수 세기 (Count Words with Given Prefix)

> 각 노드에 count 필드를 추가해 삽입 시 경로의 모든 노드 count를 증가시킵니다. 프리픽스 마지막 노드의 count가 해당 프리픽스로 시작하는 단어 수입니다.

```
final class TrieWithCount {
    final class Node {
        var children = [Character: Node]()
        var count = 0
    }
    let root = Node()
    func insert(_ word: String) {
        var cur = root
        for c in word {
            if cur.children[c] == nil { cur.children[c] = Node() }
            cur = cur.children[c]!
            cur.count += 1
        }
    }
    func countPrefix(_ prefix: String) -> Int {
        var cur = root
        for c in prefix {
            guard let n = cur.children[c] else { return 0 }
            cur = n
        }
        return cur.count
    }
}
```

시간: O(m) | 공간: O(n×m)

Medium

P3. 와일드카드 검색 (Add and Search Word)

'.'은 임의의 한 글자를 의미하므로, '.' 위치에서 모든 자식 노드를 재귀적으로 탐색합니다. DFS로 모든 경우를 확인합니다.

```
final class WildcardTrie {
    final class Node {
        var children = [Character: Node]()
        var isEnd = false
    }
    let root = Node()
    func insert(_ word: String) {
        var cur = root
        for c in word {
            if cur.children[c] == nil { cur.children[c] = Node() }
            cur = cur.children[c]!
        }
        cur.isEnd = true
    }
    func search(_ word: String) -> Bool {
        let chars = Array(word)
        func dfs(_ node: Node, _ i: Int) -> Bool {
            if i == chars.count { return node.isEnd }
            if chars[i] == "." {
                return node.children.values.contains { dfs($0, i+1) }
            }
            guard let next = node.children[chars[i]] else { return fal
            return dfs(next, i+1)
        }
        return dfs(root, 0)
    }
}
```

시간: O(m × 26^k) k=와일드카드 수 | 공간: O(n×m)

🔴 Hard

P4. 최대 XOR (Maximum XOR of Two Numbers) — Binary Trie

> 숫자를 31비트 이진수로 트라이에 저장합니다. 각 숫자에 대해 반대 비트 방향으로 탐색하면 XOR 최댓값을 구할 수 있습니다.

```
func findMaximumXOR(_ nums: [Int]) -> Int {
    var trie = [[Int]](repeating: [-1,-1], count: 32*nums.count+1)
    var sz = 1
    func insert(_ n: Int) {
        var cur = 0
        for b in stride(from: 31, through: 0, by: -1) {
            let bit = (n >> b) & 1
            if trie[cur][bit] == -1 { trie[cur][bit] = sz; sz += 1 }
            cur = trie[cur][bit]
        }
    }
    func query(_ n: Int) -> Int {
        var cur = 0, xor = 0
        for b in stride(from: 31, through: 0, by: -1) {
            let bit = (n >> b) & 1, want = 1 - bit
            if trie[cur][want] != -1 { xor |= (1 << b); cur = trie[cur
            else { cur = trie[cur][bit] }
        }
        return xor
    }
    nums.forEach { insert($0) }
    return nums.map { query($0) }.max() ?? 0
}
```

시간: O(32n) | 공간: O(32n)

CHAPTER 11 · SORTING

정렬 (Sorting)

Easy

P1. 색 정렬 (Sort Colors) — Dutch National Flag

> 세 포인터(lo, mid, hi)로 배열을 세 구역으로 분할합니다. mid가 0이면 lo와 swap 후 둘 다 전진, 2이면 hi와 swap 후 hi만 후퇴합니다.

```
func sortColors(_ nums: inout [Int]) {
    var lo = 0, mid = 0, hi = nums.count - 1
    while mid <= hi {
        switch nums[mid] {
        case 0:
            nums.swapAt(lo, mid); lo += 1; mid += 1
        case 1:
            mid += 1
        default:
            nums.swapAt(mid, hi); hi -= 1
        }
    }
}
```

시간: O(n) | 공간: O(1)

Medium

P2. 가장 큰 수 (Largest Number)

> 두 수 a, b를 이어붙인 ab와 ba를 비교하는 커스텀 comparator로 정렬합니다. 모두 "0"이면 "0"을 반환합니다.

```
func largestNumber(_ nums: [Int]) -> String {
    let sorted = nums.map(String.init).sorted {
        $0 + $1 > $1 + $0  // ab > ba 이면 a가 앞에
    }
    if sorted[0] == "0" { return "0" }
    return sorted.joined()
}
```

시간: O(n log n × m) | 공간: O(n)

Medium

P3. 상대적 순서 정렬 (Relative Sort Array)

> arr2의 원소 순서를 인덱스 맵으로 만들고, arr1을 arr2 순서 우선, 나머지는 오름차순으로 정렬합니다.

```
func relativeSortArray(_ arr1: [Int], _ arr2: [Int]) -> [Int] {
    let order = Dictionary(uniqueKeysWithValues: arr2.enumerated().map
    return arr1.sorted { a, b in
        let oa = order[a], ob = order[b]
        if let ia = oa, let ib = ob { return ia < ib }
        if oa != nil { return true }
        if ob != nil { return false }
        return a < b
    }
}
```

시간: O(n log n) | 공간: O(n)

Medium

P4. 구간 병합 (Merge Intervals)

> 시작점 기준으로 정렬 후, 현재 구간의 끝이 다음 구간의 시작보다 크거나 같으면 병합합니다. 아니면 결과에 추가하고 새 구간 시작합니다.

```
func merge(_ intervals: [[Int]]) -> [[Int]] {
    let sorted = intervals.sorted { $0[0] < $1[0] }
    var res: [[Int]] = [sorted[0]]
    for iv in sorted.dropFirst() {
        if iv[0] <= res.last![1] {
            res[res.count - 1][1] = max(res.last![1], iv[1])
        } else {
            res.append(iv)
        }
    }
    return res
}
```

시간: O(n log n) | 공간: O(n)

Hard

P5. 역전 쌍 개수 (Count of Inversions) — Merge Sort

> 병합 정렬 과정에서 오른쪽 절반의 원소가 왼쪽 절반보다 먼저 선택될 때 역전 쌍이 발생합니다.
> 병합 시 남은 왼쪽 원소 수만큼 역전이 추가됩니다.

```
func countInversions(_ nums: [Int]) -> Int {
    var arr = nums
    func mergeSort(_ a: inout [Int], _ l: Int, _ r: Int) -> Int {
        guard l < r else { return 0 }
        let m = (l + r) / 2
        var count = mergeSort(&a, l, m) + mergeSort(&a, m+1, r)
        var left = Array(a[l...m]), right = Array(a[(m+1)...r])
        var i = 0, j = 0, k = l
        while i < left.count && j < right.count {
            if left[i] <= right[j] { a[k] = left[i]; i += 1 }
            else { a[k] = right[j]; j += 1; count += left.count - i }
            k += 1
        }
        while i < left.count { a[k] = left[i]; i += 1; k += 1 }
        while j < right.count { a[k] = right[j]; j += 1; k += 1 }
        return count
    }
    return mergeSort(&arr, 0, arr.count - 1)
}
```

시간: O(n log n) | 공간: O(n)

CHAPTER 12 · BINARY SEARCH

이진 탐색 (Binary Search)

Easy

P1. 삽입 위치 찾기 (Search Insert Position) — Lower Bound

> target 이상의 첫 번째 위치(lower bound)를 이진 탐색합니다. nums[mid] < target이면 lo = mid+1, 아니면 hi = mid로 수렴합니다.

```
func searchInsert(_ nums: [Int], _ target: Int) -> Int {
    var lo = 0, hi = nums.count
    while lo < hi {
        let mid = (lo + hi) / 2
        if nums[mid] < target { lo = mid + 1 }
        else { hi = mid }
    }
    return lo
}
```

시간: O(log n) | 공간: O(1)

Easy

P2. 범위 내 원소 수 (Count of Range) — Upper/Lower Bound

> lower bound로 a 이상의 첫 위치, upper bound로 b 초과의 첫 위치를 구합니다. 그 차이가 [a, b] 범위의 원소 수입니다.

```
func countInRange(_ nums: [Int], _ a: Int, _ b: Int) -> Int {
    func lowerBound(_ target: Int) -> Int {
        var lo = 0, hi = nums.count
        while lo < hi { let m = (lo+hi)/2; nums[m] < target ? (lo=m+1)
        return lo
    }
    func upperBound(_ target: Int) -> Int {
        var lo = 0, hi = nums.count
        while lo < hi { let m = (lo+hi)/2; nums[m] <= target ? (lo=m+1
        return lo
    }
    return upperBound(b) - lowerBound(a)
}
```

시간: O(log n) | 공간: O(1)

Medium

P3. 피크 원소 찾기 (Find Peak Element)

> nums[mid] < nums[mid+1]이면 오른쪽에 피크가 있으므로 lo = mid+1로 이동합니다. 아니면 hi = mid로 이동해 수렴합니다.

```
func findPeakElement(_ nums: [Int]) -> Int {
    var lo = 0, hi = nums.count - 1
    while lo < hi {
        let mid = (lo + hi) / 2
        if nums[mid] < nums[mid + 1] { lo = mid + 1 }
        else { hi = mid }
    }
    return lo
}
```

시간: O(log n) | 공간: O(1)

🟡 Medium

P4. 회전 배열 최솟값 (Find Minimum in Rotated Sorted Array)

> nums[mid] > nums[hi]이면 최솟값이 오른쪽 절반에 있으므로 lo = mid+1, 아니면 hi = mid로 좁힙니다. lo == hi에서 수렴합니다.

```
func findMin(_ nums: [Int]) -> Int {
    var lo = 0, hi = nums.count - 1
    while lo < hi {
        let mid = (lo + hi) / 2
        if nums[mid] > nums[hi] { lo = mid + 1 }
        else { hi = mid }
    }
    return nums[lo]
}
```

시간: O(log n) | 공간: O(1)

Easy

P5. 첫 번째 불량품 (First Bad Version)

isBadVersion(mid)가 true이면 hi = mid(이 버전이 답일 수 있음), false이면 lo = mid+1로 좁힙니다. lo == hi에서 첫 번째 불량품을 찾습니다.

```
func firstBadVersion(_ n: Int) -> Int {
    var lo = 1, hi = n
    while lo < hi {
        let mid = lo + (hi - lo) / 2  // 오버플로우 방지
        if isBadVersion(mid) { hi = mid }
        else { lo = mid + 1 }
    }
    return lo
}
```

시간: O(log n) | 공간: O(1)

CHAPTER 13 · DYNAMIC PROGRAMMING

동적 프로그래밍 (DP)

Easy

P1. 집 도둑질 (House Robber)

dp[i] = max(dp[i-2] + nums[i], dp[i-1])입니다. 이전 두 값만 필요하므로 변수 두 개로 공간을 O(1)로 줄입니다.

```
func rob(_ nums: [Int]) -> Int {
    var prev2 = 0, prev1 = 0
    for n in nums {
        let cur = max(prev1, prev2 + n)
        prev2 = prev1
        prev1 = cur
    }
    return prev1
}
```

시간: O(n) | 공간: O(1)

Medium

P2. 가장 긴 증가 부분 수열 (LIS) — Patience Sorting

> tails 배열을 유지하며 이진 탐색으로 각 원소의 삽입 위치를 찾습니다. tails의 길이가 LIS 길이입니다. O(n log n) 최적 풀이입니다.

```
func lengthOfLIS(_ nums: [Int]) -> Int {
    var tails: [Int] = []
    for n in nums {
        var lo = 0, hi = tails.count
        while lo < hi {
            let mid = (lo + hi) / 2
            if tails[mid] < n { lo = mid + 1 } else { hi = mid }
        }
        if lo == tails.count { tails.append(n) }
        else { tails[lo] = n }
    }
    return tails.count
}
```

시간: O(n log n) | 공간: O(n)

Easy

P3. 유일한 경로 (Unique Paths)

dp[j]를 현재 행까지 (1행, j열)에 도달하는 경우의 수로 정의합니다. 1D DP로 공간을 O(n)으로 최적화합니다.

```
func uniquePaths(_ m: Int, _ n: Int) -> Int {
    var dp = Array(repeating: 1, count: n)
    for _ in 1..<m {
        for j in 1..<n {
            dp[j] += dp[j-1]
        }
    }
    return dp[n-1]
}
```

시간: O(m×n) | 공간: O(n)

🟡 Medium

P4. 팰린드롬 부분문자열 수 (Count Palindromic Substrings) — 중심 확장

> 각 위치를 중심으로 홀수/짝수 길이 팰린드롬을 확장합니다. 중심이 2n-1개이므로 $O(n^2)$ 시간, $O(1)$ 공간입니다.

```
func countSubstrings(_ s: String) -> Int {
    let chars = Array(s); var count = 0
    func expand(_ l: Int, _ r: Int) {
        var l = l, r = r
        while l >= 0 && r < chars.count && chars[l] == chars[r] {
            count += 1; l -= 1; r += 1
        }
    }
    for i in 0..<chars.count {
        expand(i, i)      // 홀수 길이
        expand(i, i + 1)  // 짝수 길이
    }
    return count
}
```

시간: $O(n^2)$ | 공간: $O(1)$

Medium

P5. 0/1 배낭 문제 (0/1 Knapsack)

> dp[w]를 무게 w 이하에서의 최대 가치로 정의합니다. 뒤에서부터 갱신해야 같은 아이템이 두 번 사용되지 않습니다.

```
func knapsack(_ weights: [Int], _ values: [Int], _ cap: Int) -> Int {
    var dp = Array(repeating: 0, count: cap + 1)
    for i in 0..<weights.count {
        for w in stride(from: cap, through: weights[i], by: -1) {
            dp[w] = max(dp[w], dp[w - weights[i]] + values[i])
        }
    }
    return dp[cap]
}
```

시간: O(n×W) | 공간: O(W)

Hard

P6. 풍선 터트리기 (Burst Balloons) — 구간 DP

> 구간 [l, r]에서 마지막으로 터트리는 풍선 k를 기준으로 분할합니다. dp[l][r] = max(dp[l][k-1] + nums[l-1]*nums[k]*nums[r+1] + dp[k+1][r])입니다.

```
func maxCoins(_ nums: [Int]) -> Int {
    var arr = [1] + nums + [1]
    let n = arr.count
    var dp = Array(repeating: Array(repeating: 0, count: n), count: n)
    for len in 2..<n {
        for l in 0..<(n - len) {
            let r = l + len
            for k in (l+1)..<r {
                dp[l][r] = max(dp[l][r],
                    dp[l][k] + arr[l]*arr[k]*arr[r] + dp[k][r])
            }
        }
    }
    return dp[0][n-1]
}
```

시간: $O(n^3)$ | 공간: $O(n^2)$

CHAPTER 14 · GREEDY

그리디 (Greedy)

Easy

P1. 콘텐츠 어린이 (Assign Cookies)

아이 욕구와 과자 크기를 정렬합니다. 가장 작은 과자로 가장 작은 욕구를 채우는 그리디 전략으로 최대 어린이 수를 구합니다.

```
func findContentChildren(_ g: [Int], _ s: [Int]) -> Int {
    let g = g.sorted(), s = s.sorted()
    var child = 0, cookie = 0
    while child < g.count && cookie < s.count {
        if s[cookie] >= g[child] { child += 1 }
        cookie += 1
    }
    return child
}
```

시간: O(n log n) | 공간: O(1)

Medium

P2. 점프 게임 II (Jump Game II)

> 현재 점프로 도달 가능한 최대 범위(curEnd)와 다음 점프의 최대 도달 범위(farthest)를 추적합니다. curEnd에 닿으면 점프 수를 늘립니다.

```
func jump(_ nums: [Int]) -> Int {
    var jumps = 0, curEnd = 0, farthest = 0
    for i in 0..<nums.count - 1 {
        farthest = max(farthest, i + nums[i])
        if i == curEnd {
            jumps += 1
            curEnd = farthest
        }
    }
    return jumps
}
```

시간: O(n) | 공간: O(1)

Medium

P3. 주유소 (Gas Station)

> 전체 gas 합 < cost 합이면 불가능합니다. 탱크가 0 이하가 되면 시작점을 다음으로 리셋하고 탱크를 초기화합니다.

```
func canCompleteCircuit(_ gas: [Int], _ cost: [Int]) -> Int {
    var total = 0, tank = 0, start = 0
    for i in 0..<gas.count {
        let diff = gas[i] - cost[i]
        total += diff; tank += diff
        if tank < 0 { start = i + 1; tank = 0 }
    }
    return total >= 0 ? start : -1
}
```

시간: O(n) | 공간: O(1)

Medium

P4. 태스크 스케줄러 (Task Scheduler)

> 가장 빈번한 태스크 빈도(maxFreq)와 그 개수(maxCount)로 최소 시간을 계산합니다. (maxFreq-1)*(n+1)+maxCount가 공식이며, 실제 태스크 수와 최댓값을 취합니다.

```
func leastInterval(_ tasks: [Character], _ n: Int) -> Int {
    var freq = [Int](repeating: 0, count: 26)
    for t in tasks { freq[Int(t.asciiValue! - 65)] += 1 }
    let maxFreq = freq.max()!
    let maxCount = freq.filter { $0 == maxFreq }.count
    return max(tasks.count, (maxFreq - 1) * (n + 1) + maxCount)
}
```

시간: O(n) | 공간: O(1)

Medium

P5. 최대 회의 수 (Activity Selection)

> 끝 시간 기준으로 정렬하고, 이전 회의 끝 시간 이후에 시작하는 회의를 탐욕적으로 선택합니다. Activity Selection Problem의 기본 풀이입니다.

```
func maxMeetings(_ start: [Int], _ end: [Int]) -> Int {
    let meetings = zip(start, end).sorted { $0.1 < $1.1 }
    var count = 0, lastEnd = -1
    for (s, e) in meetings {
        if s > lastEnd { count += 1; lastEnd = e }
    }
    return count
}
```

시간: O(n log n) | 공간: O(n)

CHAPTER 15 · BACKTRACKING

백트래킹 (Backtracking)

Easy

P1. 부분 집합 (Subsets)

> 각 원소에 대해 포함/미포함 두 가지 선택을 재귀적으로 탐색합니다. pick/skip 패턴으로 2^n개의 부분 집합을 생성합니다.

```
func subsets(_ nums: [Int]) -> [[Int]] {
    var res: [[Int]] = [], path: [Int] = []
    func bt(_ i: Int) {
        res.append(path)
        for j in i..<nums.count {
            path.append(nums[j])
            bt(j + 1)
            path.removeLast()
        }
    }
    bt(0)
    return res
}
```

시간: O(2^n) | 공간: O(n)

Medium

P2. 순열 (Permutations)

> 배열의 각 위치에서 나머지 원소들과 swap한 후 재귀, 복귀 시 다시 swap합니다. in-place swap으로 추가 visited 배열이 필요 없습니다.

```
func permute(_ nums: [Int]) -> [[Int]] {
    var res: [[Int]] = [], nums = nums
    func bt(_ start: Int) {
        if start == nums.count { res.append(nums); return }
        for i in start..<nums.count {
            nums.swapAt(start, i)
            bt(start + 1)
            nums.swapAt(start, i)  // 복원
        }
    }
    bt(0)
    return res
}
```

시간: O(n × n!) | 공간: O(n)

Medium

P3. 팰린드롬 분할 (Palindrome Partitioning)

> 시작 인덱스에서 끝 인덱스까지 모든 부분문자열을 시도하며, 팰린드롬이면 path에 추가하고 재귀합니다. 끝에 도달하면 결과에 추가합니다.

```
func partition(_ s: String) -> [[String]] {
    let chars = Array(s); var res: [[String]] = [], path: [String] = [
    func isPalin(_ l: Int, _ r: Int) -> Bool {
        var l = l, r = r
        while l < r { if chars[l] != chars[r] { return false }; l += 1
        return true
    }
    func bt(_ start: Int) {
        if start == chars.count { res.append(path); return }
        for end in start..<chars.count {
            if isPalin(start, end) {
                path.append(String(chars[start...end]))
                bt(end + 1)
                path.removeLast()
            }
        }
    }
    bt(0)
    return res
}
```

시간: O(2^n × n) | 공간: O(n)

Hard

P4. N-Queens (N-퀸)

> 행 단위로 퀸을 놓고, 열/대각선/역대각선 충돌을 Set으로 확인합니다. 유효한 위치에만 재귀하고 복귀 시 Set에서 제거합니다.

```
func solveNQueens(_ n: Int) -> [String] {
    // 반환 형태를 [String]이 아닌 [[String]]으로 수정
    var res: [[String]] = []
    var cols = Set<Int>(), diag = Set<Int>(), antiDiag = Set<Int>()
    var board = Array(repeating: Array(repeating: ".", count: n), coun
    func bt(_ row: Int) {
        if row == n {
            res.append(board.map { $0.joined() }); return
        }
        for col in 0..<n {
            if cols.contains(col) || diag.contains(row-col) || antiDia
            cols.insert(col); diag.insert(row-col); antiDiag.insert(ro
            board[row][col] = "Q"
            bt(row + 1)
            board[row][col] = "."
            cols.remove(col); diag.remove(row-col); antiDiag.remove(ro
        }
    }
    bt(0)
    return res.first ?? []  // 전체 solutions: return res
}
```

시간: $O(n!)$ | 공간: $O(n^2)$

Hard

P5. 다른 방법으로 계산 (Different Ways to Add Parentheses)

> 연산자를 기준으로 좌우를 분할 정복합니다. 각 부분의 결과를 조합해 모든 가능한 결과를 반환합니다. 메모이제이션으로 중복 계산을 방지합니다.

```
func diffWaysToCompute(_ expression: String) -> [Int] {
    var memo: [String: [Int]] = [:]
    func solve(_ expr: String) -> [Int] {
        if let cached = memo[expr] { return cached }
        if let n = Int(expr) { return [n] }
        var res: [Int] = []
        for (i, c) in expr.enumerated() {
            guard "+-*".contains(c) else { continue }
            let left = solve(String(expr.prefix(i)))
            let right = solve(String(expr.suffix(expr.count - i - 1)))
            for l in left { for r in right {
                switch c {
                case "+": res.append(l + r)
                case "-": res.append(l - r)
                default:  res.append(l * r)
                }
            }}
        }
        memo[expr] = res
        return res
    }
    return solve(expression)
}
```

시간: O(2^n) | 공간: O(2^n)

CHAPTER 16 · SHORTEST PATH

최단 경로 (Shortest Path)

🟡 Medium

P1. 이진 행렬 최단 경로 (Shortest Path in Binary Matrix) — BFS 8방향

> BFS로 시작점(0,0)에서 끝점(n-1, n-1)까지 8방향으로 탐색합니다. BFS는 가중치 없는 그래프에서 최단 경로를 보장합니다.

```
func shortestPathBinaryMatrix(_ grid: [[Int]]) -> Int {
    let n = grid.count
    guard grid[0][0] == 0 && grid[n-1][n-1] == 0 else { return -1 }
    if n == 1 { return 1 }
    var grid = grid
    var queue = [(0, 0, 1)]  // (row, col, distance)
    grid[0][0] = 1
    let dirs = [(-1,-1),(-1,0),(-1,1),(0,-1),(0,1),(1,-1),(1,0),(1,1)]
    var qi = 0
    while qi < queue.count {
        let (r, c, d) = queue[qi]; qi += 1
        for (dr, dc) in dirs {
            let nr = r + dr, nc = c + dc
            guard nr >= 0 && nr < n && nc >= 0 && nc < n && grid[nr][n
            if nr == n-1 && nc == n-1 { return d + 1 }
            grid[nr][nc] = 1
            queue.append((nr, nc, d + 1))
        }
    }
    return -1
}
```

시간: $O(n^2)$ | 공간: $O(n^2)$

🟡 Medium

P2. K번 이내 환승 최저가 (Cheapest Flights Within K Stops) — Bellman-Ford

> Bellman-Ford를 K+1번 반복합니다. 매 반복에서 이전 단계의 거리 배열을 복사해 같은 라운드 내 갱신이 전파되지 않도록 합니다.

```
func findCheapestPrice(_ n: Int, _ flights: [[Int]],
                       _ src: Int, _ dst: Int, _ k: Int) -> Int {
    var dist = Array(repeating: Int.max, count: n)
    dist[src] = 0
    for _ in 0...k {
        var tmp = dist
        for f in flights {
            let u = f[0], v = f[1], w = f[2]
            if dist[u] != Int.max && dist[u] + w < tmp[v] {
                tmp[v] = dist[u] + w
            }
        }
        dist = tmp
    }
    return dist[dst] == Int.max ? -1 : dist[dst]
}
```

시간: O(K × E) | 공간: O(V)

🔴 Hard

P3. 최소 노력 경로 (Path With Minimum Effort) — Dijkstra

> Dijkstra 변형으로, 가중치를 합산 대신 최대값으로 취합니다. 우선순위 큐 대신 정렬 배열로 시뮬레이션합니다. effort[r][c]가 최소화됩니다.

```
func minimumEffortPath(_ heights: [[Int]]) -> Int {
    let m = heights.count, n = heights[0].count
    var effort = Array(repeating: Array(repeating: Int.max, count: n),
    effort[0][0] = 0
    // (effort, row, col) 힙 시뮬레이션 (정렬 배열)
    var heap = [(0, 0, 0)]
    let dirs = [(0,1),(0,-1),(1,0),(-1,0)]
    while !heap.isEmpty {
        heap.sort { $0.0 < $1.0 }
        let (e, r, c) = heap.removeFirst()
        if r == m-1 && c == n-1 { return e }
        if e > effort[r][c] { continue }
        for (dr, dc) in dirs {
            let nr = r+dr, nc = c+dc
            guard nr >= 0 && nr < m && nc >= 0 && nc < n else { contin
            let newEffort = max(e, abs(heights[nr][nc] - heights[r][c]
            if newEffort < effort[nr][nc] {
                effort[nr][nc] = newEffort
                heap.append((newEffort, nr, nc))
            }
        }
    }
    return effort[m-1][n-1]
}
```

시간: O(m×n log(m×n)) | 공간: O(m×n)

🟡 Medium

P4. 임계 거리 내 도시 (Find the City) — Floyd-Warshall

> Floyd-Warshall로 모든 쌍 최단 거리를 계산합니다. 각 도시에서 distanceThreshold 이하로 도달 가능한 도시 수가 가장 적은 도시(번호 큰 것 우선)를 반환합니다.

```
func findTheCity(_ n: Int, _ edges: [[Int]], _ distanceThreshold: Int)
    var dist = Array(repeating: Array(repeating: Int.max/2, count: n),
    for i in 0..<n { dist[i][i] = 0 }
    for e in edges { dist[e[0]][e[1]] = e[2]; dist[e[1]][e[0]] = e[2]
    for k in 0..<n {
        for i in 0..<n {
            for j in 0..<n {
                if dist[i][k] + dist[k][j] < dist[i][j] {
                    dist[i][j] = dist[i][k] + dist[k][j]
                }
            }
        }
    }
    var ans = 0, minCount = n
    for i in 0..<n {
        let cnt = dist[i].filter { $0 <= distanceThreshold }.count - 1
        if cnt <= minCount { minCount = cnt; ans = i }
    }
    return ans
}
```

시간: $O(n^3)$ | 공간: $O(n^2)$

🔴 Hard

P5. K번째 최단 경로 (K-th Shortest Path) — Dijkstra + 방문 횟수

> 각 노드를 최대 k번 방문할 수 있도록 허용합니다. 노드를 k번째 방문할 때의 거리가 k번째 최단 경로입니다. visited[node] = 방문 횟수로 추적합니다.

```
func kthShortestPath(_ n: Int, _ edges: [[Int]],
                     _ src: Int, _ dst: Int, _ k: Int) -> Int {
    var adj = Array(repeating: [(Int, Int)](), count: n + 1)
    for e in edges { adj[e[0]].append((e[1], e[2])) }
    var visitCount = Array(repeating: 0, count: n + 1)
    // (distance, node) 정렬 배열로 min-heap 시뮬레이션
    var heap = [(0, src)]
    while !heap.isEmpty {
        heap.sort { $0.0 < $1.0 }
        let (d, u) = heap.removeFirst()
        visitCount[u] += 1
        if u == dst && visitCount[u] == k { return d }
        if visitCount[u] > k { continue }
        for (v, w) in adj[u] {
            if visitCount[v] < k {
                heap.append((d + w, v))
            }
        }
    }
    return -1
}
```

시간: O(k × E log(kE)) | 공간: O(kV)